



AMERICAN UNIVERSITY OF PHNOM PENH

COCS-221: Computer Science B

Lab 6: Tree & Heap Structures and Hash Tables

1. Lab Objectives

By completing this lab, you will be able to:

- **Conceptual Understanding:** Understand the structural properties, traversal algorithms, and applications of non-linear data structures including Trees, Heaps, and Hash Tables.
- **Tree Data Structures:** Implement a multi-way file system hierarchy using a binary tree via the Left-Child Right-Sibling (LCRS) representation.
- **Priority Queues & Heaps:** Build a priority-based job scheduling system using a Min-Heap and Python's built-in `heapq` library.
- **Hash Tables & Security:** Implement a credential authentication manager using a dictionary-based Hash Table and cryptographic SHA-256 hashing.
- **Software Engineering:** Develop interactive, menu-driven command-line interfaces with robust input validation, boundary checking, and cross-platform terminal compatibility.

2. Overview & Conceptual Foundations

2.1 Binary Trees and File System Representation

A binary tree is a hierarchical data structure where each node has at most two children: a left child and a right child. Binary trees are extensively used in expression parsing, search trees, and hierarchical data modeling.

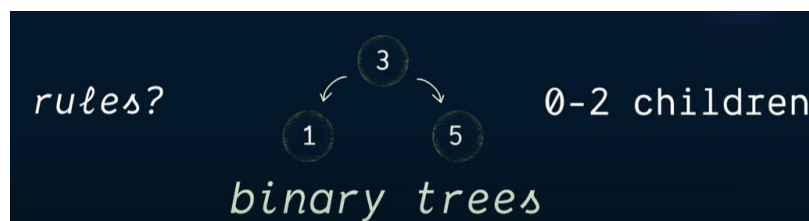


Figure 1: Fundamental Binary Tree Structure (0 to 2 children per node)

Key Concept: Left-Child Right-Sibling (LCRS) Representation

Real operating system file systems (such as Windows NTFS or Linux ext4) are multi-way trees: a single folder can contain zero, two, or dozens of subfolders and files. To represent an arbitrary multi-way directory hierarchy using a binary tree structure, we use the Left-Child Right-Sibling (LCRS) convention:

- left pointer: Points to the first child (the first subfolder or file contained within this directory).
- right pointer: Points to the next sibling (another file or folder residing at the same hierarchical level).

This elegant representation allows a strictly binary node structure (left, right) to represent directories with unlimited child entries.

2.2 Heaps and Priority Queues

A heap is a specialized complete binary tree that satisfies the heap ordering property:

- **Max-Heap:** The key of the parent node is always greater than or equal to the keys of its children. The largest element is always at the root.
- **Min-Heap:** The key of the parent node is always smaller than or equal to the keys of its children. The smallest element is always at the root.

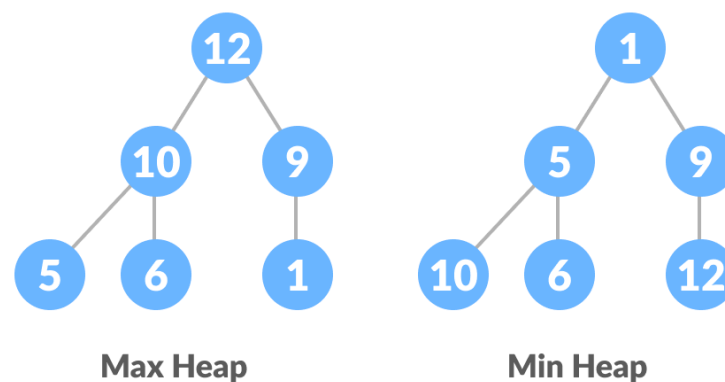
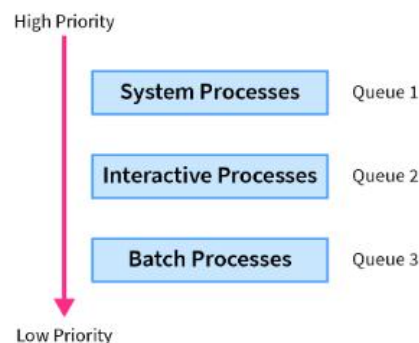


Figure 2: Max-Heap vs. Min-Heap Properties

In operating systems, heaps form the backbone of Priority Queues. Rather than serving processes purely first-come-first-served (FIFO), urgent tasks (such as hardware interrupts or antivirus scanners) are assigned a lower priority number and scheduled before background processes.



2.3 Hash Tables and Cryptographic Hashing

A hash table stores key-value pairs and computes an array index via a hash function, providing average $O(1)$ time complexity for insertion, search, and deletion. In Python, the built-in dict type is an optimized hash table.

In modern security architectures, passwords must never be stored in plain text. Instead, passwords are run through a one-way cryptographic hash function (such as SHA-256). The hash is irreversible, meaning even system administrators cannot reconstruct the plain-text password from the stored hash.

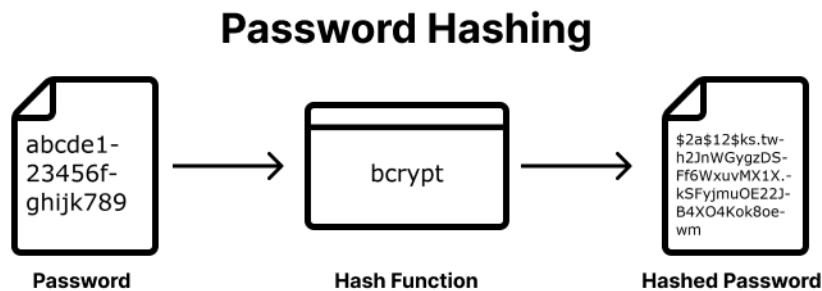


Figure 4: Secure One-Way Password Hashing Workflow

Important: Windows Terminal / PowerShell UTF-8 Encoding Tip

If you are developing on Windows and run your Python scripts in Command Prompt or PowerShell, your default terminal encoding may be cp1252. Printing emojis (such as 📁, 📄, ✅, ❌) may raise a `UnicodeEncodeError`.

To safely prevent this crash across all environments (including IDLE, VS Code, and command prompt), add this check at the top of your scripts:

```
import sys
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8')
```

Note: In Python IDLE or Thonny, `sys.stdout` is a GUI object that already handles Unicode natively, so `hasattr(...)` ensures it won't raise an `AttributeError`.

Alternatively, you may use standard ASCII symbols such as [DIR], [FILE], [OK], and [X].

3. Task 1: Binary Trees for File System Management

Every modern operating system manages files and folders in a hierarchical tree. In this task, you will implement a simulated file system explorer using an LCRS binary tree.

3.1 Requirements & Specifications

- **Node Class:** Define a Node class where each node contains: name (str), is_file (bool), left (pointer to first child), and right (pointer to next sibling).
- **add_node:** Implement add_node(parent, name, is_file=False) to dynamically insert a file or directory under a specified parent directory.
 - Enforce integrity rules:
 - a) You cannot add a file or directory inside a file (parent.is_file must be False).
 - b) Unique names: No two items (files or directories) in the same directory may share the exact same name.
 - c) Return True if addition succeeded, or False if the item already exists.
- **search:** Implement search(node, name) to search recursively for a file or directory by name starting from the root using Depth-First Search (DFS). Return the Node object if found, or None if not found.
- **print_tree:** Implement print_tree(node, level=0) to recursively display the directory structure with hierarchical indentation and icons (📁 for directories, 📄 for files).
- **User Interface:** Implement an interactive menu loop allowing the user to (1) Add Directory or File, (2) Print File System Structure, (3) Search for a File/Directory, and (4) Exit.

3.2 Function Signatures & Starter Code Guide

```
class Node:
    def __init__(self, name: str, is_file: bool = False):
        self.name = name
        self.is_file = is_file
        self.left = None    # First child (subdirectory or file)
        self.right = None   # Sibling directory/file

    def add_node(parent: Node, name: str, is_file: bool = False) -> bool:
        """Adds a new file or directory under parent. Returns False if name exists."""
        # TODO: Check if name already exists among parent's children.
        # TODO: Append new_node to the end of the sibling linked list.
        pass

    def search(node: Node, name: str) -> Node | None:
        """Recursively searches for a node by name in the tree."""
        # TODO: Implement DFS search over left (child) and right (sibling) pointers.
        pass

    def print_tree(node: Node, level: int = 0) -> None:
        """Prints the hierarchical structure with indentation."""
        # TODO: Print current node with indentation, then recurse on left and right.
        pass
```

3.3 Expected Output

Your program should produce console output matching the reference interaction below:

```

1. Add Directory or File
2. Print File System Structure
3. Search for a File/Directory
4. Exit
Enter your choice (1/2/3/4): 1
Enter the parent directory name: Root
Enter the name of the file or directory: Documents
Is it a file or directory? (f/d): d
Added directory 'Documents' under 'Root'.

Choose an action:
1. Add Directory or File
2. Print File System Structure
3. Search for a File/Directory
4. Exit
Enter your choice (1/2/3/4): 2

File System Structure:
  Root
    Documents

Choose an action:
1. Add Directory or File
2. Print File System Structure
3. Search for a File/Directory
4. Exit
Enter your choice (1/2/3/4): 3
Enter the name of the file or directory to search: Documents
Found: Documents (Directory)

Choose an action:
1. Add Directory or File
2. Print File System Structure
3. Search for a File/Directory
4. Exit
Enter your choice (1/2/3/4): 4
Exiting program.

```

Figure 5: Task 1 Expected Output Interaction

4. Task 2: Heaps for Job Scheduling (Priority Queue)

Operating systems schedule processes using priority queues so that urgent tasks execute before lower-priority background tasks. In this task, you will implement a `JobScheduler` using a Min-Heap.

4.1 Requirements & Specifications

- **JobScheduler Class:** Define a `JobScheduler` class with an internal list `job_queue` managed as a min-heap using Python's `heapq` module.
- **add_job:** `add_job(job_name, priority)`: Adds a job to the queue. In a min-heap, smaller priority numbers represent higher urgency (e.g., priority 1 runs before priority 3).
- **execute_job:** `execute_job()`: Removes and executes the job with the highest priority (lowest number) using `heapq.heappop`. If the queue is empty, print 'No jobs in the queue!'.

- **show_jobs:** show_jobs(): Displays all current jobs in the queue sorted by priority without emptying or destroying the heap.
- **Input Validation:** Input Validation: When prompting for job priority, ensure the user enters a valid integer. If invalid input is provided, handle the ValueError and repeatedly prompt until a valid integer is entered.
- **User Interface:** Provide an interactive menu with options: 1. Add Job, 2. Execute Job, 3. Show Jobs, 4. Exit.

Programming Tip: Tie-Breaking in Tuples

When storing tuples like (priority, job_name) in heapq, if two jobs have identical priorities, Python compares the second element (job_name strings). To preserve First-In, First-Out (FIFO) ordering for jobs with identical priorities, you can include an auto-incrementing sequence counter: (priority, counter, job_name).

4.2 Starter Code Guide

```
import heapq

class JobScheduler:
    def __init__(self):
        self.job_queue = [] # Min-heap list storing (priority, job_name)

    def add_job(self, job_name: str, priority: int) -> None:
        heapq.heappush(self.job_queue, (priority, job_name))
        print(f"Job '{job_name}' with priority {priority} added!")

    def execute_job(self) -> None:
        # TODO: Check if queue is empty; pop and execute highest priority job.
        pass

    def show_jobs(self) -> None:
        # TODO: Display jobs sorted by priority without emptying the queue.
        pass
```

4.3 Expected Output

Your program should produce console output matching the reference interactions below:

```

1. Add Job
2. Execute Job
3. Show Jobs
4. Exit
Enter choice: 1
Enter job name: JobA
Enter job priority (lower number = higher priority): 3
Job 'JobA' with priority 3 added!

1. Add Job
2. Execute Job
3. Show Jobs
4. Exit
Enter choice: 1
Enter job name: JobB
Enter job priority (lower number = higher priority): 1
Job 'JobB' with priority 1 added!

1. Add Job
2. Execute Job
3. Show Jobs
4. Exit
Enter choice: 3

Current Job Queue:
- JobB (Priority: 1)
- JobA (Priority: 3)

1. Add Job
2. Execute Job
3. Show Jobs
4. Exit
Enter choice: 2
Executing job: JobB (Priority: 1)

```

Figure 6: Task 2 Adding and Executing Jobs

```

1. Add Job
2. Execute Job
3. Show Jobs
4. Exit
Enter choice: 4
Exiting program.
PS C:\Users\TOP>

```

Figure 7: Task 2 Clean Exit

5. Task 3: Hash Tables for Secure Password Management

Websites and cloud systems protect user accounts by storing cryptographic hashes rather than raw passwords. In this task, you will implement a user credential authentication system using a Hash Table (Python dictionary) and SHA-256.

5.1 Requirements & Specifications

- **PasswordManager Class:** Define a PasswordManager class with an internal dictionary `user_data` to map `username` -> `sha256_hash`.

- **hash_password:** hash_password(self, password): Uses hashlib.sha256(password.encode('utf-8')).hexdigest() to return the 64-character hexadecimal digest.
- **register_user:** register_user(): Prompts for username and password. If username already exists, print an error message. Otherwise, store the hashed password and print a success message.
- **login_user:** login_user(): Prompts for username and password. Hashes the entered password and compares it against the stored hash. Prints welcome message on success or incorrect password message on failure.
- **Menu & Feedback:** Interactive Menu: Must match the exact menu options and status messages specified in Figure 8:
 - Option 1: 1. Register User
 - Option 2: 2. Login User
 - Option 3: 3. Exit
 - Choice Prompt: Enter your choice:
 - Registration Success: ☒ User '<username>' registered successfully.
 - Login Failure: ☒ Incorrect password!
 - Login Success: ☒ Login successful! Welcome, <username>.
 - Exit Message: Exiting program.

5.2 Starter Code Guide

```
import hashlib

class PasswordManager:
    def __init__(self):
        self.user_data = {} # Hash table mapping username -> sha256_hash

    def hash_password(self, password: str) -> str:
        return hashlib.sha256(password.encode('utf-8')).hexdigest()

    def register(self, username: str, password: str) -> bool:
        # TODO: Return False if username exists, else store hash and return True.
        pass

    def login(self, username: str, password: str) -> bool:
        # TODO: Verify username exists and compare password hash.
        pass
```

5.3 Expected Output

Your program should produce console output matching the reference interaction below:

```

1. Register User
2. Login User
3. Exit
Enter your choice: 1
Enter a username: john
Enter a password: john99
✅ User 'john' registered successfully.

1. Register User
2. Login User
3. Exit
Enter your choice: 1
Enter a username: lisa
Enter a password: lisa20
✅ User 'lisa' registered successfully.

1. Register User
2. Login User
3. Exit
Enter your choice: 2
Enter your username: john
Enter your password: john20
❌ Incorrect password!

1. Register User
2. Login User
3. Exit
Enter your choice: 2
Enter your username: lisa
Enter your password: lisa20
✅ Login successful! Welcome, lisa.

1. Register User
2. Login User
3. Exit
Enter your choice: 3
Exiting program.

```

Figure 8: Task 3 User Registration and Login Flow

— End of Lab 6 Assignment —